

LABORATORIO 05

La columna que no existía

Guía para leer mientras trabajas el cuaderno. Cada celda trae la pregunta que responde, la idea en el almacén, la sentencia explicada parte por parte, lo que sale en pantalla y cómo leerlo. Las salidas son las de la solución ejecutada del repositorio.

Introducción

1 · El tema

El formato de los documentos tributarios cambió. Los que llegan desde 2026 traen dos datos que antes no existían, el canal por el que se emitió el documento y la sucursal. Los de 2024 no los tienen y nunca los van a tener. En este laboratorio vas a cambiar la forma de una tabla que ya tiene datos, agregando, renombrando, ensanchando y quitando columnas, y vas a comprobar que lo guardado no se toca.

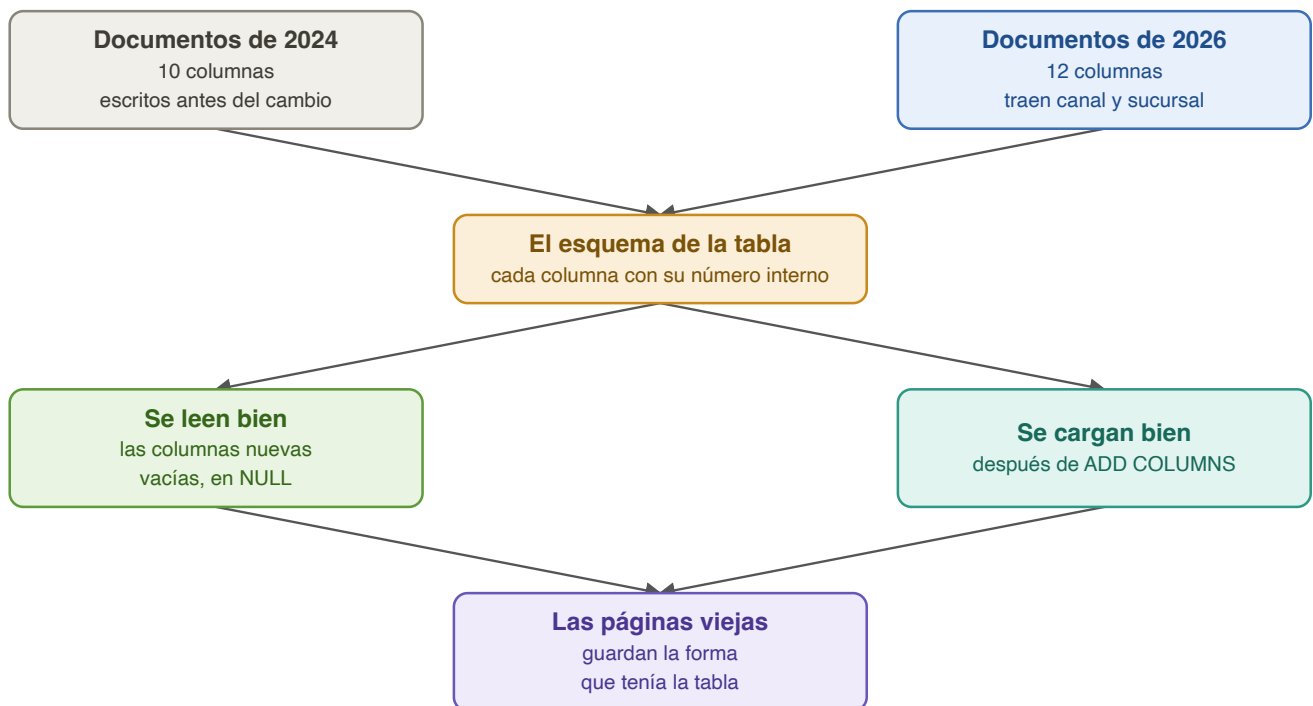
2 · El problema, en el almacén

Desde marzo el almacenero también anota el teléfono de cada proveedor. Las páginas viejas no tienen esa casilla. Si para agregarla tuviera que reescribir toda la libreta, no lo haría nunca. Y si al agregar la casilla leyera las páginas viejas contando casillas desde la izquierda, empezaría a leer el monto donde dice el estado, y la libreta entera quedaría corrida sin que nadie lo notara.

3 · El problema en términos técnicos

Cambiar el esquema de una tabla, sus columnas y sus tipos, suele exigir reprocesar la historia para que la tabla acepte el formato nuevo. Son semanas de trabajo y el riesgo de romper lo que ya funcionaba. Peor todavía, en tablas que leen las columnas por posición, agregar una columna puede dejar los valores viejos corridos de lugar, en silencio.

4 · El diagrama



Documentos de 2024 (gris). Los que ya estaban en la tabla, escritos con diez columnas.

Documentos de 2026 (azul). El lote nuevo, con doce columnas. Trae `canal_emision` y `codigo_sucursal`.

El esquema de la tabla (amarillo). La descripción de las columnas, guardada en la portada. Iceberg le da a cada columna un número interno propio, que no cambia aunque la columna se renombre o se mueva. Así identifica las columnas, no por su posición ni por su nombre.

Se leen bien (verde). Los documentos viejos se siguen leyendo sin error. Las columnas que no tenían aparecen vacías, en `NULL`.

Se cargan bien (verde agua). Una vez agregadas las columnas, el lote nuevo entra con la misma sentencia que antes fallaba.

Las páginas viejas (morado). Cada página de la libreta recuerda la forma que tenía la tabla cuando se escribió, y por eso se puede volver a ver la tabla como era.

5 · La solución

En el almacén, el almacenero agrega la casilla del teléfono en la portada de la libreta y sigue anotando. Las páginas viejas quedan como estaban, con esa casilla en blanco. Él las lee sin confundirse porque conoce cada casilla por su nombre y no cuenta posiciones. Si un día la casilla teléfono pasa a llamarse contacto, cambia el nombre en la portada y ninguna página se reescribe.

En Iceberg eso es la **evolución de esquema**. `ALTER TABLE` agrega, renombra, ensancha o quita columnas cambiando solo la descripción de la tabla, sin reescribir ningún archivo de datos. Cada columna tiene un número interno que no cambia, así que los archivos viejos se leen siempre bien, y cada página guarda el esquema con que se escribió.

6 · Los pasos

- **Paso 0.** Creas la tabla con un año de documentos de 2024 y miras su esquema de diez columnas.
- **Paso 1.** Miras el lote de 2026, de doce columnas, e intentas cargarlo tal cual, y falla a propósito.
- **Paso 2.** Agregas las dos columnas y compruebas que los documentos viejos siguen intactos.
- **Paso 3.** Cargas el lote de 2026 y miras las dos épocas juntas.
- **Paso 4.** Renombras una columna, ensanchas un tipo y quitas una columna, comprobando cada vez.
- **Paso 5.** Miras el registro de versiones del esquema y viajas a la primera página, donde la columna todavía no existía.
- **Cierre.** Los cuatro cambios de forma y lo que se reescribió en cada uno.

La celda 0.2 borra la tabla si existe, así que el laboratorio se puede repetir desde arriba. Si prefieres dejar todo limpio antes de empezar, corre el comando de abajo desde la carpeta del repositorio.

```
bash
bin/reiniciar-lab.sh 05
```

Paso 0 · Preparar la tabla

Celda 0.1 · Ponerte en tu espacio

La pregunta. ¿En qué espacio de trabajo va a quedar la tabla?

Por qué ahora. La tabla del laboratorio tiene que quedar en tu espacio, `mi_espacio`. Esta celda te pone ahí.

En el almacén. Es pararte frente a tu propio mostrador antes de abrir la libreta.

La sentencia, parte por parte.

- `%%sql` es la primera línea de toda celda SQL del cuaderno. Le dice al cuaderno que lo que sigue es SQL y no Python.
- `-- Celda 0.1` es un comentario. Todo lo que va después de dos guiones lo ignora el motor.
- `USE mi_espacio` deja a `mi_espacio` como espacio de trabajo.

```
sql
%%sql
-- Celda 0.1
USE mi_espacio
```

Lo que sale en pantalla.

```
text
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/09/25 18:22:41 WARN NativeCodeLoader: Unable to load native-hadoop library for your
platform... using builtin-java classes where applicable
Listo. La sentencia se ejecutó.
```

Cómo se lee. Ya estás en tu espacio. `USE` no devuelve filas, por eso el cuaderno solo avisa que se ejecutó.

Las tres primeras líneas no son errores. Salen solo en la primera celda que usa Spark, cuando arranca.

- `Setting default log level to "WARN"` dice que Spark va a mostrar solo los avisos y los errores, no todo lo que hace.
- `To adjust logging level use sc.setLogLevel(newLevel)` explica cómo cambiar eso. `sc` es el contexto de Spark y `setLogLevel` la función que cambia el nivel. No hace falta tocarlo. SparkR es la versión de Spark para el lenguaje R, que aquí no se usa.
- `WARN NativeCodeLoader: Unable to load native-hadoop library` dice que Spark no encontró una biblioteca de Hadoop escrita para este sistema operativo y que usa la versión en Java. Funciona igual. `26/09/25 18:22:41` es la fecha y la hora del aviso, año 2026, mes 09, día 25, en hora UTC.

Celda 0.2 · Partir de cero

La pregunta. ¿Qué pasa si la tabla ya existe de una vez anterior?

Por qué ahora. Si el laboratorio se corrió antes, la creación de la celda siguiente fallaría. Esta celda borra la tabla primero.

En el almacén. Es sacar la libreta vieja de este ejercicio, si quedó alguna.

La sentencia, parte por parte.

- `DROP TABLE` borra la tabla.
- `IF EXISTS` hace que no falle si no existe.
- `documentos_lab05` es la tabla del laboratorio, en `mi_espacio`.

```
sql
%%sql
-- Celda 0.2
DROP TABLE IF EXISTS documentos_lab05
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. Se ejecutó sin problema, existiera o no la tabla.

Celda 0.3 · Crear la tabla con el formato antiguo

La pregunta. ¿Cómo se crea una tabla con los documentos de 2024, tal como llegaban antes del cambio?

Por qué ahora. Es la tabla con historia sobre la que se va a cambiar la forma.

En el almacén. Es abrir una libreta nueva y copiar en ella un año de facturas con el formato de siempre.

La sentencia, parte por parte.

- `CREATE TABLE documentos_lab05` crea la tabla.
- `USING iceberg` hace que sea una tabla Iceberg.
- `AS SELECT * FROM curso.dte_2024` le da las columnas de esa consulta y la carga. `curso.dte_2024` es un año de documentos de 2024, en el espacio `curso` de las tablas de referencia.

```
sql
%%sql
-- Celda 0.3
CREATE TABLE documentos_lab05 USING iceberg AS
SELECT * FROM curso.dte_2024
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. La tabla quedó creada y cargada, y dejó su primera página en la libreta.

Celda 0.4 · Contar los documentos

La pregunta. ¿Cuántos documentos tiene la tabla?

Por qué ahora. Es el número que tiene que seguir igual después de cada cambio de forma.

En el almacén. Es contar las facturas de la libreta.

La sentencia, parte por parte.

- `SELECT count(*)` cuenta todas las filas.
- `AS documentos` le pone nombre a la columna.
- `FROM documentos_lab05` es tu tabla.

```
sql
%%sql
-- Celda 0.4
SELECT count(*) AS documentos FROM documentos_lab05
```

Lo que sale en pantalla.

```
text
documentos
30000
1 fila.
```

Cómo se lee. 30000 documentos, un año completo.

Celda 0.5 · La forma de la tabla

La pregunta. ¿Qué columnas tiene la tabla y de qué tipo es cada una?

Por qué ahora. Esa lista es el esquema, y es lo que se va a cambiar. Conviene verla antes.

En el almacén. Es leer en la portada qué casillas tiene cada página.

La sentencia, parte por parte.

- `DESCRIBE TABLE` pide las columnas de la tabla con sus tipos.
- `documentos_lab05` es tu tabla.

```
sql
%%sql
-- Celda 0.5
DESCRIBE TABLE documentos_lab05
```

Lo que sale en pantalla.

```
text
col_name      data_type      comment
rut_emisor    string
razon_social_emisor string
tipo_dte      int
folio         bigint
fecha_emision  date
fecha_recepcion timestamp
monto_netto   decimal(18,2)
monto_iva     decimal(18,2)
monto_total   decimal(18,2)
estado_sii    string

# Partitioning
Not partitioned
13 filas.
```

Cómo se lee. Diez columnas. Esa es la forma de la tabla con el formato antiguo.

- `col_name` es el nombre de la columna, `data_type` su tipo y `comment` un comentario, que aquí está vacío.
- `string` es texto. `int` es un número entero de hasta unos dos mil millones y `bigint` un entero mucho más grande. `date` es una fecha, `timestamp` fecha con hora, y `decimal(18,2)` un número de hasta 18 dígitos con 2 decimales, que es como se guardan los montos.
- `# Partitioning` y `Not partitioned` dicen que la tabla no está particionada. Esas dos filas y la fila vacía de arriba cuentan en las `13 filas.` del pie, que son diez columnas más tres filas de separación.

Paso 1 · Llega el formato nuevo

Celda 1.1 · La forma del lote de 2026

La pregunta. ¿Qué forma tiene el lote nuevo?

Por qué ahora. Antes de cargar un lote se mira su forma. Así se sabe si calza con la tabla.

En el almacén. Es mirar las casillas de las facturas nuevas antes de copiarlas.

La sentencia, parte por parte.

- `DESCRIBE TABLE` pide las columnas con sus tipos.
- `curso.dte_2026_reciente` es el lote de 2026, en el espacio `curso`.

```
sql
%%sql
-- Celda 1.1
DESCRIBE TABLE curso.dte_2026_reciente
```

Lo que sale en pantalla.

```
text
col_name      data_type      comment
rut_emisor    string
razon_social_emisor string
tipo_dte      int
folio         bigint
fecha_emision date
fecha_recepcion timestamp
monto_neto    decimal(18,2)
monto_iva     decimal(18,2)
monto_total   decimal(18,2)
estado_sii    string
canal_emision string
codigo_sucursal string

# Partitioning
Not partitioned
15 filas.
```

Cómo se lee. Doce columnas. Las diez de siempre y, al final, dos que tu tabla no tiene, `canal_emision` y `codigo_sucursal`, las dos de texto.

Celda 1.2 · Intentar cargarlo tal cual

La pregunta. ¿Qué hace el motor si le llega un lote con más columnas que la tabla?

Por qué ahora. Es lo que haría cualquiera. Esta celda falla a propósito, y el error es parte de lo que hay que saber.

En el almacén. Es intentar copiar facturas de doce casillas en una libreta que solo tiene diez.

La sentencia, parte por parte.

- `INSERT INTO documentos_lab05` agrega filas a tu tabla.
- `SELECT * FROM curso.dte_2026_reciente` son todas las filas del lote de 2026, con todas sus columnas.

```
sql
%%sql
-- Celda 1.2
INSERT INTO documentos_lab05 SELECT * FROM curso.dte_2026_reciente
```

Lo que sale en pantalla.

```
text
Spark rechazó la sentencia:

Cannot write to 'spark_catalog.mi_espacio.documentos_lab05', too many data columns:
Table columns: 'rut_emisor', 'razon_social_emisor', 'tipo_dte', 'folio', 'fecha_emision',
'fecha_recepcion', 'monto_neto', 'monto_iva', 'monto_total', 'estado_sii'
Data columns: 'rut_emisor', 'razon_social_emisor', 'tipo_dte', 'folio', 'fecha_emision',
'fecha_recepcion', 'monto_neto', 'monto_iva', 'monto_total', 'estado_sii', 'canal_emision',
'codigo_sucursal'
```

Cómo se lee. Falló, y no entró nada. `Spark rechazó la sentencia` es el aviso del cuaderno cuando el motor devuelve un error.

- `Cannot write to 'spark_catalog.mi_espacio.documentos_lab05', too many data columns` dice que no puede escribir en la tabla porque los datos traen demasiadas columnas. `spark_catalog` es el nombre del catálogo.
- `Table columns` lista las diez columnas de la tabla.
- `Data columns` lista las doce que traen los datos.

Lo importante es lo que no hizo. No descartó columnas en silencio, no metió los valores corridos y no cargó nada a medias. Falló entero, y eso es una buena noticia para quien opera la tabla.

Paso 2 · Agregar las columnas

Celda 2.1 · Agregar las dos columnas

La pregunta. ¿Cómo se agranda la tabla para el formato nuevo?

Por qué ahora. El error dijo exactamente qué faltaba. Se agregan las dos columnas en una sola sentencia.

En el almacén. Es agregar dos casillas en la portada de la libreta. Las páginas escritas no se tocan.

La sentencia, parte por parte.

- `ALTER TABLE documentos_lab05` modifica la tabla.
- `ADD COLUMNS` agrega columnas al final.
- `(canal_emision STRING, codigo_sucursal STRING)` son las dos columnas, cada una con su tipo, separadas por coma.

```
sql
%%sql
-- Celda 2.1
ALTER TABLE documentos_lab05
ADD COLUMNS (canal_emision STRING, codigo_sucursal STRING)
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. Se hizo al instante. No se movió ningún dato. Lo único que cambió es la descripción de la tabla.

Celda 2.2 · Leer los documentos viejos

La pregunta. Los documentos de 2024 se escribieron con diez columnas. ¿Qué pasa si se leen ahora que la tabla tiene doce?

Por qué ahora. Es la prueba de que agregar columnas no corrompe lo que ya estaba.

En el almacén. Es leer una página vieja después de agregar las casillas nuevas en la portada.

La sentencia, parte por parte.

- `SELECT rut_emisor, folio, monto_total, canal_emision, codigo_sucursal` pide tres columnas viejas y las dos nuevas.
- `FROM documentos_lab05` es tu tabla.
- `ORDER BY rut_emisor, tipo_dte, folio` ordena para que salga siempre igual.
- `LIMIT 3` se queda con tres filas.

```
sql
%%sql
-- Celda 2.2
SELECT rut_emisor, folio, monto_total, canal_emision, codigo_sucursal
FROM documentos_lab05
ORDER BY rut_emisor, tipo_dte, folio
LIMIT 3
```

Lo que sale en pantalla.

```
text
rut_emisor folio monto_total canal_emision codigo_sucursal
76000262-3 3 11401907.65 None None
76000262-3 42 6219295.81 None None
76000262-3 62 2852995.25 None None
3 filas.
```

Cómo se lee. Los datos viejos están intactos, el RUT, el folio y el monto en su lugar. Las dos columnas nuevas dicen `None`.

`None` es como el cuaderno muestra un `NULL`, que en SQL significa que no hay valor. Es la respuesta correcta, porque esos documentos de 2024 nunca tuvieron canal de emisión ni sucursal. No se perdió nada, simplemente no existía.

Iceberg no se confunde porque cada archivo de datos guarda sus columnas con el número interno de cada una. Al leer, busca en cada archivo la columna por su número. Las columnas nuevas tienen números que los archivos viejos no conocen, así que salen vacías, y las viejas se encuentran siempre donde están.

Celda 2.3 · Confirmar que no se perdió nada

La pregunta. ¿Siguen todos los documentos, y cuántos tienen canal?

Por qué ahora. Para confirmar con números que el cambio no tocó los datos.

En el almacén. Es contar las facturas y cuántas tienen escrita la casilla nueva.

La sentencia, parte por parte.

- `SELECT count(*) AS documentos` cuenta todas las filas.
- `count(canal_emision) AS con_canal` cuenta solo las filas donde `canal_emision` tiene valor. `count` de una columna no cuenta los `NULL`.
- `FROM documentos_lab05` es tu tabla.

```
sql
%%sql
-- Celda 2.3
SELECT count(*)          AS documentos,
       count(canal_emision) AS con_canal
FROM documentos_lab05
```

Lo que sale en pantalla.

```
text
documentos con_canal
30000      0
1 fila.
```

Cómo se lee. 30000 documentos y ninguno con canal. Todo en su lugar.

Paso 3 · Cargar el lote nuevo

Celda 3.1 · La misma sentencia de antes

La pregunta. ¿Entra ahora el lote de 2026?

Por qué ahora. Es la misma sentencia que falló en la celda 1.2, sin cambiarle una letra. Ahora la tabla ya tiene las doce columnas.

En el almacén. Es copiar las facturas nuevas en la libreta que ya tiene las casillas que les faltaban.

La sentencia, parte por parte.

- `INSERT INTO documentos_lab05` agrega filas a tu tabla.
- `SELECT * FROM curso.dte_2026_reciente` son todas las filas del lote de 2026.

```
sql
%%sql
-- Celda 3.1
INSERT INTO documentos_lab05 SELECT * FROM curso.dte_2026_reciente
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. Entró. Esta carga deja la segunda página de la libreta.

Celda 3.2 · Las dos épocas juntas

La pregunta. ¿Cómo conviven los documentos viejos y los nuevos?

Por qué ahora. Para ver en una sola consulta los dos formatos en la misma tabla.

En el almacén. Es contar las facturas por año, y cuántas de cada año tienen la casilla nueva escrita.

La sentencia, parte por parte.

- `SELECT year(fecha_emision) AS anio` saca el año de la fecha de emisión. `year` es la función que devuelve el año.
- `count(*) AS documentos` cuenta los documentos de cada año.
- `count(canal_emision) AS con_canal` cuenta los que tienen canal.
- `FROM documentos_lab05` es tu tabla.
- `GROUP BY year(fecha_emision)` junta las filas por año.
- `ORDER BY anio` ordena del año más viejo al más nuevo.

```

sql
%%sql
-- Celda 3.2
SELECT year(fecha_emision) AS anio,
       count(*)           AS documentos,
       count(canal_emision) AS con_canal
FROM documentos_lab05
GROUP BY year(fecha_emision)
ORDER BY anio

```

Lo que sale en pantalla.

```

text
anio  documentos  con_canal
2024  30000         0
2026  2500          2500
2 filas.

```

Cómo se lee. En 2024 hay 30000 documentos y ninguno con canal. En 2026 hay 2500 y todos tienen canal. Dos formatos en una sola tabla, respondiendo a la misma consulta.

Paso 4 · Los otros cambios

Celda 4.1 · Renombrar una columna

La pregunta. ¿Cómo se cambia el nombre de una columna?

Por qué ahora. Agregar no es el único cambio. `codigo_sucursal` es un nombre incómodo y conviene llamarla `sucursal`.

En el almacén. Es cambiar el nombre de la casilla en la portada.

La sentencia, parte por parte.

- `ALTER TABLE documentos_lab05` modifica la tabla.
- `RENAME COLUMN codigo_sucursal TO sucursal` cambia el nombre de la columna. El número interno de la columna no cambia.

```
sql
%%sql
-- Celda 4.1
ALTER TABLE documentos_lab05 RENAME COLUMN codigo_sucursal TO sucursal
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. El nombre cambió al instante, sin reescribir ningún archivo.

Celda 4.2 · El dato bajo el nombre nuevo

La pregunta. ¿Sigue el dato ahí, ahora con otro nombre?

Por qué ahora. Si la tabla buscara las columnas por su nombre, renombrar dejaría huérfanos los datos ya escritos. Hay que comprobarlo.

En el almacén. Es leer la casilla con su nombre nuevo en las facturas que ya estaban.

La sentencia, parte por parte.

- `SELECT rut_emisor, folio, canal_emision, sucursal` pide la columna con su nombre nuevo.
- `FROM documentos_lab05` es tu tabla.
- `WHERE canal_emision IS NOT NULL` se queda con los documentos que sí tienen canal, los de 2026. `IS NOT NULL` significa que tiene valor.
- `ORDER BY rut_emisor, tipo_dte, folio` ordena para que salga siempre igual.
- `LIMIT 3` se queda con tres filas.

```
sql
%%sql
-- Celda 4.2
SELECT rut_emisor, folio, canal_emision, sucursal
FROM documentos_lab05
WHERE canal_emision IS NOT NULL
ORDER BY rut_emisor, tipo_dte, folio
LIMIT 3
```

Lo que sale en pantalla.

```
text
rut_emisor  folio  canal_emision  sucursal
76000262-3  33     API_DGT        SUC-022
76003963-2  1      API_DGT        SUC-031
76005022-9  43     API_DGT        SUC-003
3 filas.
```

Cómo se lee. El mismo dato, con otro nombre. `API_DGT` es el canal por el que se emitieron y `SUC-022`, `SUC-031` y `SUC-003` son códigos de sucursal. Los archivos escritos con el nombre viejo se leen bien porque Iceberg busca la columna por su número interno, no por su nombre.

Celda 4.3 · Ensanchar un tipo

La pregunta. ¿Se puede cambiar el tipo de una columna que ya tiene datos?

Por qué ahora. Si la DGT va a usar códigos de tipo de documento más largos, la columna `tipo_dte` tiene que aceptarlos.

En el almacén. Es hacer más ancha una casilla para que quepan números más grandes.

La sentencia, parte por parte.

- `ALTER TABLE documentos_lab05` modifica la tabla.
- `ALTER COLUMN tipo_dte TYPE BIGINT` cambia el tipo de `tipo_dte` de `int` a `bigint`, de entero a entero grande.

```
sql
%%sql
-- Celda 4.3
ALTER TABLE documentos_lab05 ALTER COLUMN tipo_dte TYPE BIGINT
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. Se hizo sin reescribir nada. Todo número que cabía en un `int` cabe en un `bigint`, así que los valores guardados se leen igual.

Celda 4.4 · Los valores después de ensanchar

La pregunta. ¿Siguen bien los valores que ya estaban?

Por qué ahora. Para confirmarlo con los datos.

En el almacén. Es contar las facturas de cada tipo después de ensanchar la casilla.

La sentencia, parte por parte.

- `SELECT tipo_dte, count(*) AS documentos` muestra cada tipo y cuántos documentos tiene.
- `FROM documentos_lab05` es tu tabla.
- `GROUP BY tipo_dte` junta por tipo.
- `ORDER BY tipo_dte` ordena de menor a mayor.

```
sql
%%sql
-- Celda 4.4
SELECT tipo_dte, count(*) AS documentos
FROM documentos_lab05
GROUP BY tipo_dte
ORDER BY tipo_dte
```

Lo que sale en pantalla.

```
text
tipo_dte  documentos
33        13722
34         2305
39        12310
52         1904
56         637
61        1622
6 filas.
```

Cómo se lee. Seis tipos de documento, todos en su lugar. 33 es factura afecta, 34 factura exenta, 39 boleta, 52 guía de despacho, 56 nota de débito y 61 nota de crédito. Sumados dan los treinta y dos mil quinientos documentos de la tabla.

Al revés no se puede. Pasar de `bigint` a `int` podría no caber, y el motor lo rechaza con el mensaje `bigint cannot be cast to int`.

Celda 4.5 · Quitar una columna

La pregunta. ¿Cómo se elimina una columna que nadie usa?

Por qué ahora. Al final el código de sucursal no lo usaba nadie.

En el almacén. Es borrar la casilla de la portada. Las páginas escritas no se tocan.

La sentencia, parte por parte.

- `ALTER TABLE documentos_lab05` modifica la tabla.
- `DROP COLUMN sucursal` quita la columna del esquema.

```
sql
%%sql
-- Celda 4.5
ALTER TABLE documentos_lab05 DROP COLUMN sucursal
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. La columna ya no está en la tabla. Los archivos no se reescribieron, así que el espacio que ocupaba ese dato no se libera todavía.

Celda 4.6 · El resto, intacto

La pregunta. ¿Quedó bien el resto de la tabla?

Por qué ahora. Quitar una columna no puede tocar las demás.

En el almacén. Es leer las facturas nuevas después de borrar la casilla.

La sentencia, parte por parte.

- `SELECT rut_emisor, folio, canal_emision` pide tres columnas que siguen en la tabla.
- `FROM documentos_lab05` es tu tabla.
- `WHERE canal_emision IS NOT NULL` se queda con los documentos de 2026.
- `ORDER BY rut_emisor, tipo_dte, folio` ordena.
- `LIMIT 3` se queda con tres filas.

```
sql
%%sql
-- Celda 4.6
SELECT rut_emisor, folio, canal_emision
FROM documentos_lab05
WHERE canal_emision IS NOT NULL
ORDER BY rut_emisor, tipo_dte, folio
LIMIT 3
```

Lo que sale en pantalla.

```
text
rut_emisor folio canal_emision
76000262-3 33 API_DGT
76003963-2 1 API_DGT
76005022-9 43 API_DGT
3 filas.
```

Cómo se lee. Los mismos tres documentos de la celda 4.2, sin la columna `sucursal`. Lo demás sigue igual.

Paso 5 · La historia del esquema

Celda 5.1 · Las versiones de la portada

La pregunta. ¿Deja la tabla algún registro de sus cambios de forma?

Por qué ahora. Cuatro cambios de forma y ningún archivo reescrito. Queda ver qué anotó la libreta.

En el almacén. Cada vez que cambia algo, el almacenero escribe una portada nueva. Esta celda lista todas las portadas.

La sentencia, parte por parte.

- `SELECT timestamp` es la hora en que se escribió cada portada, en UTC.
- `latest_schema_id AS version_del_esquema` es el número de esquema de la página vigente en ese momento.
- `FROM mi_espacio.documentos_lab05.metadata_log_entries` lee la vista de sistema `.metadata_log_entries`, una fila por cada `metadata.json` que tuvo la tabla. El nombre va con tres partes.
- `ORDER BY timestamp` ordena de la más vieja a la más nueva.

```
sql
%%sql
-- Celda 5.1
SELECT timestamp, latest_schema_id AS version_del_esquema
FROM mi_espacio.documentos_lab05.metadata_log_entries
ORDER BY timestamp
```

Lo que sale en pantalla.

```
text
timestamp          version_del_esquema
2026-09-25 18:22:51.043  0
2026-09-25 18:22:52.330  0
2026-09-25 18:22:53.222  1
2026-09-25 18:22:53.907  1
2026-09-25 18:22:54.401  1
2026-09-25 18:22:54.860  1
6 filas.
```

Cómo se lee. Seis portadas, una por cada cambio en la tabla. La columna de la derecha pasa de 0 a 1. El 0 es el formato antiguo, de diez columnas. El 1 es la tabla con las columnas nuevas.

El detalle de cada fila.

- 18:22:51.043 UTC, las 15:22 en Chile, es la creación de la tabla con el esquema 0.
- 18:22:52.330 UTC es el `ADD COLUMNS`. Todavía dice 0 porque la columna muestra el esquema de la última página con datos, y esa página seguía siendo la de la creación.
- 18:22:53.222 UTC es la carga de 2026, que escribió una página con el esquema 1.
- Las tres portadas siguientes son el renombre, el cambio de tipo y el borrado de la columna. Siguen diciendo 1 porque ninguno de esos cambios escribió una página de datos.

Por eso ese número no avanza con cada cambio, y la lista completa de cambios no se lee de ahí. Lo que sí se puede hacer es ver la tabla como era, y eso es lo que sigue.

Celda 5.2 · Las páginas de la libreta

La pregunta. ¿Qué páginas tiene la libreta?

Por qué ahora. Para volver a la tabla como estaba antes de todos los cambios, hace falta el número de la primera página.

En el almacén. Es mirar la lista de páginas escritas.

La sentencia, parte por parte.

- `SELECT snapshot_id, committed_at, operation` pide el número de cada página, la hora en que se colgó, en UTC, y qué se hizo.
- `FROM mi_espacio.documentos_lab05.snapshots` es la vista de sistema con la lista de páginas.
- `ORDER BY committed_at` ordena de la más vieja a la más nueva.

```
sql
%%sql
-- Celda 5.2
SELECT snapshot_id, committed_at, operation
FROM mi_espacio.documentos_lab05.snapshots
ORDER BY committed_at
```

Lo que sale en pantalla.

```
text
snapshot_id      committed_at      operation
1387296462588068198  2026-09-25 18:22:51.043  append
1146300458587039073  2026-09-25 18:22:53.222  append
2 filas.
```

Cómo se lee. Dos páginas, las dos `append`. La primera, `1387296462588068198`, es la carga de 2024. La segunda, `1146300458587039073`, es la de 2026. Los cuatro cambios de forma no dejaron página, porque no escribieron datos.

Celda 5.3 · Viajar a la primera página

La pregunta. ¿Cómo estaba la tabla antes de que llegara el formato nuevo?

Por qué ahora. La libreta guarda sus páginas, y cada página recuerda la forma que tenía la tabla.

En el almacén. Es abrir la libreta en la primera página y contar las facturas que había.

La sentencia, parte por parte.

- `SELECT count(*) AS documentos` cuenta las filas.
- `FROM documentos_lab05` es tu tabla.
- `VERSION AS OF <TU_SNAPSHOT_ID>` pide la tabla como estaba en esa página. `<TU_SNAPSHOT_ID>` es un marcador. Donde está, escribes sin comillas el `snapshot_id` de la primera fila de la celda 5.2. En la solución fue `1387296462588068198`.

```
sql
%%sql
-- Celda 5.3
SELECT count(*) AS documentos FROM documentos_lab05 VERSION AS OF <TU_SNAPSHOT_ID>
```

Lo que sale en pantalla.

```
text
documentos
30000
1 fila.
```

Cómo se lee. 30000 documentos, la tabla como estaba antes de la carga de 2026.

Celda 5.4 · Pedir la columna que no existía

La pregunta. Si en esa página la columna `canal_emision` todavía no existía, ¿qué pasa si la pido?

Por qué ahora. Esta celda falla a propósito, y es la última.

En el almacén. Es buscar la casilla del teléfono en una página escrita antes de que la casilla existiera.

La sentencia, parte por parte.

- `SELECT canal_emision` pide la columna nueva.
- `FROM documentos_lab05` es tu tabla.
- `VERSION AS OF <TU_SNAPSHOT_ID>` es la misma primera página de la celda anterior, con el mismo número.
- `LIMIT 1` pide una sola fila.

```
sql
%%sql
-- Celda 5.4
SELECT canal_emision FROM documentos_lab05 VERSION AS OF <TU_SNAPSHOT_ID> LIMIT 1
```

Lo que sale en pantalla.

```
text
Spark rechazó la sentencia:

Column 'canal_emision' does not exist. Did you mean one of the following?
[spark_catalog.mi_espacio.documentos_lab05.folio,
spark_catalog.mi_espacio.documentos_lab05.tipo_dte,
spark_catalog.mi_espacio.documentos_lab05.monto_iva,
spark_catalog.mi_espacio.documentos_lab05.estado_sii,
spark_catalog.mi_espacio.documentos_lab05.monto_netto,
spark_catalog.mi_espacio.documentos_lab05.rut_emisor,
spark_catalog.mi_espacio.documentos_lab05.fecha_emision,
spark_catalog.mi_espacio.documentos_lab05.monto_total,
spark_catalog.mi_espacio.documentos_lab05.fecha_recepcion,
spark_catalog.mi_espacio.documentos_lab05.razon_social_emisor]; line 2 pos 7;
```

Cómo se lee. `Column 'canal_emision' does not exist`, la columna no existe. En esa página de la libreta la casilla todavía no estaba. La tabla no solo recuerda sus datos, también la forma que tenía cuando los guardó.

- `Did you mean one of the following?` significa ¿quisiste decir alguna de estas?, y lista las diez columnas que sí existían en esa página, con su nombre completo, catálogo, espacio, tabla y columna.
- `line 2 pos 7` dice dónde está el error en la sentencia, en la línea 2, posición 7. La línea 1 es el comentario de la celda, y en la línea 2 la posición 7 es donde empieza `canal_emision`.

Preguntas frecuentes

¿Por qué None y no un espacio en blanco?

Porque `None` es como el cuaderno muestra un `NULL`, la ausencia de valor. Un texto vacío sería un valor, y diría que el documento se emitió por un canal sin nombre. `NULL` dice que el dato no existe, que es lo que pasa con los documentos de 2024.

¿Cómo sabe Iceberg qué columna es cuál si cambio el nombre?

Porque no la busca por el nombre ni por la posición. Cada columna tiene un número interno que se asigna al crearla y no cambia nunca. Los archivos de datos guardan ese número, y al leer Iceberg traduce el número al nombre que tenga hoy la columna.

¿Agregar una columna reescribe los archivos?

No. Ninguno de los cuatro cambios del laboratorio reescribe datos. Solo cambia la descripción de la tabla en la portada. Por eso son instantáneos, con treinta mil documentos o con doscientos millones.

¿Por qué no se liberó espacio al quitar la columna?

Porque los archivos viejos no se reescribieron y todavía guardan ese dato. El espacio se recupera cuando la tabla se compacta y se botan las páginas viejas, no al hacer el `ALTER`.

¿Se puede achicar un tipo?

No. Se puede ensanchar, de `int` a `bigint` o de un decimal a uno con más dígitos, porque todo lo que cabía antes cabe después. Al revés un valor podría no caber, y el motor responde `bigint cannot be cast to int`.

¿Se puede reordenar las columnas?

En este ambiente, no. `ALTER COLUMN ... AFTER` y `... FIRST` fallan contra el catálogo, con un error que además avisa que no puede saber si el cambio se aplicó. No lo intentes en una tabla que te importe. En la práctica se resuelve eligiendo las columnas en el orden que quieras al consultar.

¿De dónde saco el número que va en <TU_SNAPSHOT_ID>?

De la salida de la celda 5.2, la columna `snapshot_id` de la primera fila, la de la carga de 2024. Se escribe sin comillas, y el mismo número sirve para las celdas 5.3 y 5.4. El de la solución fue `1387296462588068198`, pero el tuyo es otro.

¿Por qué las dos celdas que fallan no rompen la tabla?

Porque un error no escribe nada. La celda 1.2 falló antes de escribir y la 5.4 es solo una lectura. La tabla queda exactamente como estaba.

Cierre

La pregunta. ¿Qué te llevas de este laboratorio?

Que cambiar la forma de una tabla no pone en riesgo lo que ya está guardado. Los documentos de 2024 sobrevivieron a cuatro cambios de esquema, se siguen leyendo bien, y las columnas que nunca tuvieron salen vacías.

En el almacén. El almacenero cambió las casillas de la portada cuatro veces sin reescribir una sola página, y cada página vieja sigue diciendo lo que decía cuando se escribió.

Los cuatro cambios.

El cambio	La sentencia	¿Reescribe datos?
Agregar columnas	ALTER TABLE ... ADD COLUMNS (...)	No
Renombrar una columna	ALTER TABLE ... RENAME COLUMN a TO b	No
Ensanchar un tipo	ALTER TABLE ... ALTER COLUMN c TYPE BIGINT	No
Eliminar una columna	ALTER TABLE ... DROP COLUMN c	No

Los números de la solución ejecutada.

Momento	Documentos	Con canal
Carga de 2024	30.000	0
Después de agregar las columnas	30.000	0
Después de cargar 2026	32.500	2.500
En la primera página	30.000	la columna no existe

Un cambio de formulario ya no significa reprocesar años de historia. Es una sentencia, y la tabla recuerda la forma que tenía en cada página.